Selection Beats Compression: Memory Decision Policies for Large Language Model Agents Under a Bounded Store

YASH RAMCHANDANI[1], SIMAR SINGH KHANUJA[1], DR. PRAVEEN MODI[1,*]

[1]*School of Technology Management & Engineering, SVKM's NMIMS Deemed to be University, Indore Campus, India*

Abstract

Background. Agents built on large language models (LLMs) accumulate conversations far longer than any context window, so a memory layer must decide what to keep, compress, forget and recall. Existing memory systems are evaluated end to end, so the contribution of each decision is never measured on its own, and the usual accounting bounds the context sent to the model but leaves the store unbounded.

Methods. MemGate is an evaluation harness that sits between an agent and a frozen LLM, isolates the memory policy and compares policies on a common accuracy-per-token frontier, changing one factor at a time. It bounds the store as well as the assembled context, in tokens or in bytes with the embedding index included. Policies are evaluated on LoCoMo, REALTALK and BEAM with paired bootstrap intervals clustered by conversation. The primary metric, answer recall, checks that the answer text itself reaches the context, not only the identifier of its turn. We compare a hand-written heuristic, a 0.5B LLM judge and a logistic salience scorer over shallow features and a sentence embedding, trained on generated and human conversations together.

Results. Bounding only the context charges a store-everything baseline nothing for retention. With the store bounded, choosing which turns to forget gains +9.37 answer-recall points over oldest-first eviction at identical storage, fidelity and retrieval (95% CI $[+5.45, +14.90]$), while compressing evicted turns loses 7.13 points, with no rate–distortion crossover up to 182× compression. A 1.5B model that rewrites evicted turns narrows this gap but does not close it. Counting the embedding index reverses the ranking below about 40 KiB. The LLM judge loses to the regular-expression heuristic by 5.87 points, because salience is not a property of a turn read on its own. The learned scorer beats the heuristic by +14.55 points on held-out conversations, transfers from human to generated dialogue but not the reverse, and outperforms MemGPT-style and Mem0-style memory at every budget tested. On held-out chats of up to ten million tokens it beats retrieval on every corpus; on million-token chats typed into the application it keeps the answer for 51.9% of questions against 19.2% for retrieval, and +6.0 points of its advantage become correct answers through a fixed reader. Three results changed sign or size once a measurement error was removed, and each was caught by an invariant rather than by reading the code.

Conclusions. Selection, not compression, is what a bounded memory should spend its budget on, counted in bytes with the index included. The policy ships as an offline chat application built on the experimental code, which reproduces the retrieval and selection results at a 4096-token store (28.1% and 37.5%) exactly.

*Corresponding author: Dr. Praveen Modi, Assistant Professor, School of Technology Management & Engineering, SVKM's NMIMS Deemed to be University, Indore Campus, India.

Keywords: Large language models, agent memory, context management, retrieval-augmented generation, rate–distortion, empirical evaluation

1. Introduction

An agent that runs for weeks [1, 2] builds up a history far longer than any context window [3], so on every turn something has to decide what goes into the prompt. A bigger window does not make that decision go away. Across frontier models, accuracy drops by 30–50% well before the advertised limit [4], and how much a model uses a passage depends on where it sits in a long input [5]. Both effects come from attention over long sequences rather than from missing training data, so a larger window shifts the problem instead of removing it. What limits the agent is curation, not capacity.

Poor curation has a measurable price. On LongMemEval, oracle retrieval reaches about 92%, while the same model choosing its own evidence reaches about 58% [6]. Given the right evidence the model answers correctly, so the 34-point gap is a failure of selection, not of reasoning.

Systems that tackle selection already exist, and agent architectures now treat memory as a module in its own right [7]. MemGPT [8], MemoryBank [9] and the designs surveyed in [10] all summarise and forget across tiers. What nobody has shown is whether each of those mechanisms is worth what it costs, because the systems are evaluated whole rather than one decision at a time. The same survey puts it plainly: current systems "do not solve the fundamental challenge: deciding what to remember and what to forget" [10]. A rate–distortion view [11, 12] of these decisions has been proposed [13], but as an analytical framework, not a measured system.

We set out to measure the decision itself. Our question is simple to state: how much of the selection gap can a better memory policy close, and at what budget does compression start to pay?

We make eleven contributions. (1) A storage-budgeted protocol. Bounding only the assembled context charges a store-everything baseline nothing for what it keeps, which decides the comparison before it is run (Section 6.1). (2) Evidence that the gain comes from selection, not compression: at identical storage, fidelity and retrieval, scored forgetting is worth +9.37 answer-recall points over FIFO, while compressing an evicted turn costs 7.13 (Sections 6.2 and 6.3). (3) A test of the rate–distortion crossover up to 182× compression (Section 6.4). (4) Accounting in bytes, which shows the embedding index outweighing the text it indexes by about 12× and flipping the policy ranking below roughly 40 KiB (Section 6.5). (5) A negative result for LLM-judged salience: swapped in as a single component, a 0.5B judge loses to a hand-written heuristic by 5.87 points (Section 6.6). (6) A learned scorer trained on two corpora, one generated by an LLM and one written by people, together with the direction in which it transfers and the one in which it does not (Section 6.7). (7) An end-task evaluation through a fixed reader, showing how much of the recall advantage turns into correct answers (Section 6.8). (8) Comparisons with MemGPT-style and Mem0-style memory rebuilt in the same harness (Section 6.9). (9) A scale test on chats of up to ten million tokens, and a scorer retrained on 14.6 times the data, including the finding that more data on its own did not help (Sections 6.10 and 6.11). (10) A working application that runs the same code as the experiments, with the scoring rules and equations it uses (Sections 3.3 and 7). (11) Three measurement errors in our own results and the checks that caught them (Section 8).

Five of the eleven are negative or corrective, and we think that is the useful part. Described only as a three-tier memory layer, MemGate would repeat work others have done. Measured against controls, it tells us which parts of that design actually earn their keep.

2. Related Work

Several memory systems for LLM agents are already in use. MemGPT [8] pages a virtual context in and out of external storage under the model's own control. MemoryBank [9] adds an Ebbinghaus forgetting curve. Generative Agents [14] rank memories by recency, importance and relevance, A-MEM [15] keeps them as linked notes, and Mem0 [16] uses an LLM to extract and update facts. The shared design recalls the multi-store model of human memory [17]. Together these systems show that tiering and forgetting work in practice. Because each is evaluated end to end, though, no single decision inside them can be credited or blamed on its own, and that is the separation our harness is built to make.

A second line of work shrinks the prompt or the attention cache instead of choosing among stored items. LLMLingua [18] prunes tokens by perplexity. Gist tokens [19] and AutoCompressors [20] learn dense summaries, and the in-context autoencoder [21] packs a context into memory slots. StreamingLLM [22] and H_2O [23] evict entries from the KV cache using attention scores. Architectural memories go further and change the model, compressing [24] or indexing [25, 26] past activations. All of these work inside one forward pass over a contiguous context, or need the model to be trained. The decision we study sits upstream and persists: it fixes which items survive from one session to the next, and an item evicted there cannot be recovered from any cache.

Retrieval augmentation [27–29] is the store-everything baseline that memory systems are measured against, and it is the RAG arm of our harness. Choosing what to evict from a bounded store is also the classic page-replacement problem [30, 31]. The difference is that here the future accesses are questions that no policy gets to see.

On the evaluation side, LoCoMo [32] provides multi-session dialogues with gold evidence turns marked, which lets us compute a retrieval-level metric without running a generative model. LongMemEval [6] separates several memory abilities and is where the gap quoted above comes from, and LongBench [33] tests long inputs with no memory layer at all. REALTALK [34] applies LoCoMo's annotation scheme to 21 days of messages between real people. That makes it the natural check on whether a scorer fitted to LoCoMo learned what matters, or only what LLM-written dialogue looks like. We offer neither a new architecture nor a new benchmark. Our contribution is a measurement protocol and what it finds, including one case where fixing the protocol reverses the conclusion.

3. The MemGate Harness

MemGate sits between an agent and a frozen LLM and controls one thing: what goes into the prompt (Fig. 1). As in retrieval augmentation [27], the base model is a black box that we never train or fine-tune, so any difference we report comes from the memory policy.

3.1. Tiers as a lifecycle

The store has three tiers, but an item moves through them over its life rather than being filed into one. Every turn enters Tier 1 word for word, and the routing decision is

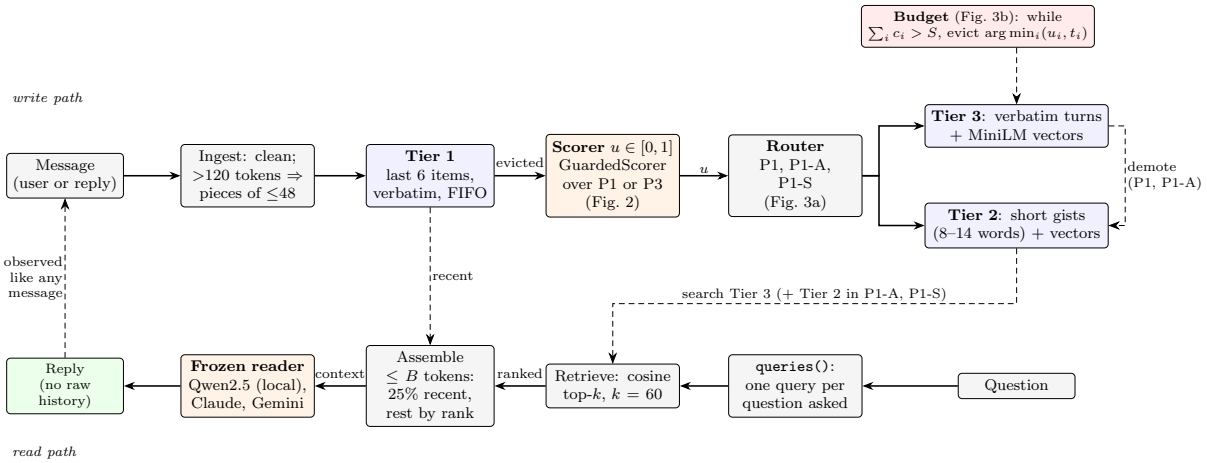


Figure 1: System architecture. Write path (top): every message, the reader's replies included, enters Tier 1 verbatim; when it leaves Tier 1 it is scored once and routed, and the budget enforcer holds the store at S. (The application adds an entry gate in front of Tier 1, Section 7; the experiments do not use it.) Read path (bottom): each question in a message is its own retrieval query, and the reader receives only the assembled context of at most B tokens, never the raw history. Dashed arrows carry state rather than the item being processed.

made once, when the turn is evicted. With u the utility the scorer gives an item, the rule is

$$\begin{aligned} u \geq \tau_{\text{fact}} &\rightarrow \text{Tier 3: atomic fact + vector} \\ u \geq \tau_{\text{low}} &\rightarrow \text{Tier 2: compressed gist} \\ \text{otherwise} &\rightarrow \text{dropped} \end{aligned} \tag{1}$$

with $\tau_{\text{fact}} = 0.45$ and $\tau_{\text{low}} = 0.15$; an item with $u \geq \tau_{\text{fact}} + 0.2$ also leaves a gist in Tier 2. The tiers follow the working-set [31] and multi-store [17] views of memory. Tier 1 holds recent verbatim turns and answers the question *when*; Tier 2 holds compressed gists and answers *gist*; Tier 3 holds durable facts with embeddings, answers *what*, and enters the context only when retrieval matches. Because each item is scored exactly once, at eviction, the decision has a single site in the pipeline, and that is what lets a controlled swap target it.

3.2. Policies

The arms are P0 (sliding window, recency, verbatim), RAG (store-all, FIFO, verbatim), P1 (threshold, scored at ingest), P1-A (adaptive: score orders eviction, compress), P1-S (select: score orders eviction, drop), P2 (0.5B judge), P3 (learned, never fitted on the conversation it scores) and an oracle. Two require comment. RAG is the no-policy control: it stores everything, retrieves the top k items, and evicts FIFO when the store is full. It is not a weak baseline but the arm that a decision policy has to beat in order to have contributed anything. P0 is given the same token budget as MemGate, so the only difference between them is which turns are selected, not how many tokens may be spent.

3.3. Scoring, routing and the storage budget

Every equation in this subsection is transcribed from the code that produced the results: the modules `scoring`, `pretrained`, `ingest`, `store` and `policies`, and the two training scripts. A message m is always scored with its speaker tag removed, and the scorer sees its text and speaker only: no question, evidence label or future turn.

The P1 heuristic. Let $W(m)$ be the words of m, D a list of 28 durability cues (*deadline, budget, prefer, born, lives, works*, ...), and $h(m) = |W(m) \cap D|$. With indicators δ_{num}

(a digit), δ_{tmp} (an amount of money, or a month prefix), δ_{prop} (a capitalised word not opening a sentence), $\delta_{?}$ (ends with a question mark), δ_{asst} (written by the assistant) and δ_{upd} (a correction cue such as *actually* or *no longer* together with a digit or month),

$$\begin{aligned} s(m) &= 0.15 + \min(0.35,\, 0.18\,h) + 0.20\,\delta_{\text{num}} + 0.15\,\delta_{\text{tmp}} \\ &\quad + 0.12\,\delta_{\text{prop}} - 0.12\,\delta_{?} - 0.10\,\delta_{\text{asst}} + 0.10\,\delta_{\text{upd}}, \\ u_{\text{P1}}(m) &= \begin{cases} 0.02, & m \text{ is filler}, \\ \min\big(1, \max(0,\, s(m))\big), & \text{otherwise}, \end{cases} \end{aligned} \tag{2}$$

where a message is filler when it is an acknowledgement or greeting from a closed list, or has at most two words. The weights were set by hand and never fitted.

The learned scorer P3. Its input concatenates 15 hand features $\phi(m)$ (word and character counts, digit, month, money, proper noun, question, final period, $h(m)$, correction cue, filler, assistant speaker, share of words of at most three letters, capital letters, clock time) with the 384-dimensional unit-norm MiniLM embedding $e(m)$ [35, 36]:

$$\begin{aligned} x(m) &= \big[\phi(m);\, e(m)\big] \in \mathbb{R}^{399}, \\ z(m) &= b + \sum_{j=1}^{399} w_j\, \frac{x_j(m) - \mu_j}{\sigma_j}, \quad u_{\text{P3}}(m) = \frac{1}{1 + e^{-z(m)}}, \end{aligned} \tag{3}$$

with μ, σ the training mean and standard deviation of each input. The label is distant supervision [37]: $y_i = 1$ when any question cites turn i as evidence. With N training turns from corpora c grouped into families $f \in F$ (LoCoMo, REALTALK, BEAM, MemBench, everyday chat), the fit is an ℓ_2-regularised, class-balanced, sample-weighted logistic regression fitted with scikit-learn [38],

$$\begin{aligned} &\min_{\mathbf{w},b}\ \tfrac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \omega_i\, \kappa_{y_i}\, \ell\big(y_i,\, u_{\text{P3}}(m_i)\big), \\ &\omega_i = \frac{N\,\pi_{c(i)}}{|F|\,n_{c(i)}}, \qquad \kappa_y = \frac{N}{2N_y}, \end{aligned} \tag{4}$$

where ℓ is the log loss, $C = 1$, n_c is the number of turns of corpus c, and π_c is the share of its family's weight that corpus c receives: n_c/n_f (by turns, the recipe of the PR #12 model), or P_c/P_f (by evidence turns P, used for the larger model). Every family therefore weighs the same, whatever its size. Generated small-talk lines enter as negatives with total weight $0.05N$.

The shipped ensemble. The shipped scorer averages the logits of two such models, A (the PR #12 recipe) and B (all 856,876 turns). Because each logit is linear in x, the mean is again a single logistic regression, which is how it is stored (Fig. 4):

$$\begin{aligned} \bar{z}(m) &= \tfrac{1}{2}\big(z_A(m) + z_B(m)\big) = \bar{b} + \bar{\mathbf{w}}^{\top} x(m), \\ \bar{w}_j &= \tfrac{1}{2} \sum_{k \in \{A,B\}} \frac{w_j^{(k)}}{\sigma_j^{(k)}}, \quad \bar{b} = \tfrac{1}{2} \sum_k \Big(b^{(k)} - \sum_j \frac{w_j^{(k)} \mu_j^{(k)}}{\sigma_j^{(k)}} \Big). \end{aligned} \tag{5}$$

P1-S only orders by u, but the thresholds of (1) and the guards below read u as absolute. The logit is therefore mapped by an affine function that leaves every ranking unchanged, in the manner of Platt scaling [39],

$$\begin{aligned} z' &= \alpha \bar{z} + \beta, \qquad \beta = \operatorname{logit} \tau_{\text{fact}} - \alpha\, q_{\text{fact}}, \\ \alpha &= \frac{\operatorname{logit} \tau_{\text{fact}} - \operatorname{logit} \tau_{\text{low}}}{q_{\text{fact}} - q_{\text{low}}}, \end{aligned} \tag{6}$$

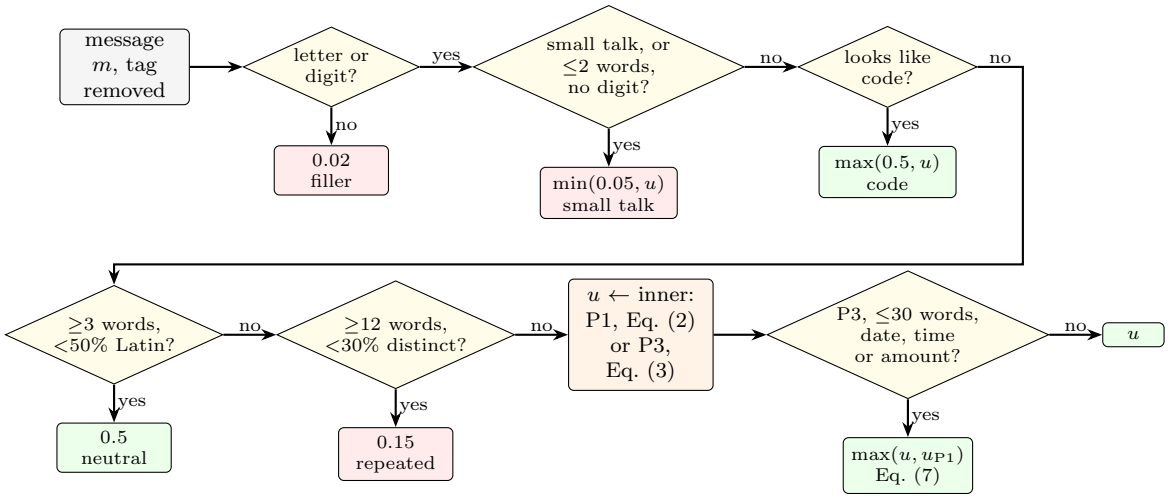


Figure 2: How the application scores a message (GuardedScorer). Guards are tried left to right; a message that none of them covers is scored by the inner scorer, and a short message naming a date, time or amount is floored at the P1 score, Eq. (7).

where q_{fact} and q_{low} are the quantiles of $\bar{z}$ over LoCoMo and REALTALK turns that put 36.48% of them above τ_{fact} and 71.85% above τ_{low}, the shares of the first shipped model.

The application's guard. The application wraps its scorer in GuardedScorer (Fig. 2). Each guard fires on one kind of input the scorers were not built for (no words, small talk, code, non-Latin script, repetition), and ordinary English passes straight through. One further rule corrects a measured weakness of the learned scorer: trained with BEAM, whose long answers are full of numbers nobody asks about, it gave a short dated fact the score of a greeting. For a message of $n(m) \le 30$ words that names a month, a clock time or an amount,

$$u_{\text{G}}(m) = \max\big(u_{\text{P3}}(m),\, u_{\text{P1}}(m)\big), \tag{7}$$

and $u_{\text{G}} = u_{\text{P3}}$ otherwise. The experiments never use the guard.

The storage budget. Each stored item i has a cost

$$c_i = \begin{cases} \text{tok}(i) & \text{(tokens)}, \\ \text{bytes}(i) + 1536\mathbb{1}[i \text{ has a vector}] & \text{(bytes)}, \end{cases} \tag{8}$$

and after every write the store enforces

$$\sum_{i \in T_1 \cup T_2 \cup T_3} c_i \;\le\; S \tag{9}$$

by evicting, while it is violated,

$$v = \arg\min_{i \in T_3} (u_i,\, t_i) \tag{10}$$

in lexicographic order, t_i being the turn index, so the oldest item breaks a tie (Fig. 3b). Superseded records are reclaimed before any eviction. P1 and P1-A demote v to a Tier 2 gist of at most eight words when that gist costs less than v, and drop it otherwise; P1-S always drops it; when Tier 3 is empty the same rule is applied to Tier 2. Tier 1, the six-item live buffer, is never evicted. RAG uses $\arg\min_i t_i$, first in first out, and has no scorer. With equal costs, repeated application of (10) leaves P1-S holding exactly

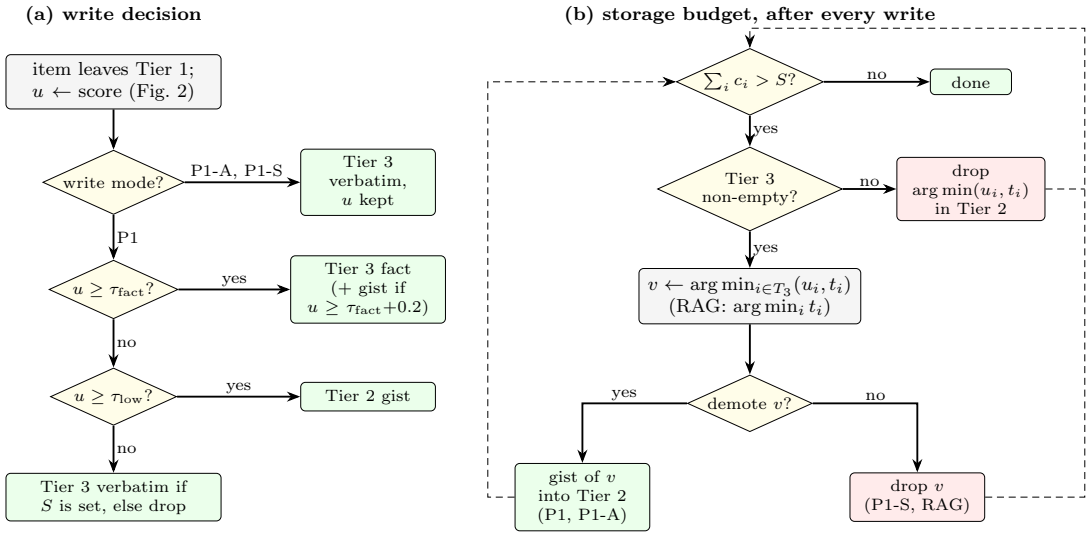


Figure 3: Where the keep, compress and forget decisions are taken. (a) Routing of an item leaving Tier 1, Eq. (1): threshold mode (P1) decides at ingest; the adaptive modes (P1-A, P1-S) keep everything verbatim and let u order the eviction queue. (b) The budget loop of Eqs. (9)–(10), run after every write. P1-A and P1-S differ only at the "demote" decision, and P1-S and RAG only in the eviction key.

the highest-utility items seen so far, and RAG the newest ones, the FIFO rule of page replacement [30]; this is the only difference between them.

Context assembly. For a question q and context budget B (2048 tokens in every experiment), the most recent Tier 1 items are packed into $\lfloor B/4 \rfloor$ tokens. In P1-A and P1-S the remainder is filled greedily from one pool of Tier 2 and Tier 3 items,

$$R_k(q) = \text{top}_k \left\{ i : \cos\big(e(q), e(i)\big) > 0 \right\}, \qquad k = 60, \tag{11}$$

ranked by cosine similarity. P1 instead gives Tier 2 a fixed quarter of B, packed by information density, and fills the last half from Tier 3 by (11). When a message asks several questions, each is its own query and the rankings are interleaved, best of each first.

Store maintenance. Three rules keep the store consistent, and none reads an evaluation label. A write to Tier 3 whose embedding has cos ≥ 0.92 with an active fact is folded into that fact instead of being stored twice. A write the P1 heuristic labels a correction (Section 3.3: a correction cue with a digit or month) retires the single most similar active fact with cos ≥ 0.60. When Tier 2 outgrows its share, its oldest half is consolidated: items with $u \geq 0.5$ are promoted to Tier 3 verbatim and the rest are merged into one coarser gist.

3.4. One variable per comparison

The experimental design lies in the set of contrasts rather than in the set of policies. Each contrast holds everything constant except one factor:

Contrast	Held constant	Varies
P0 vs. RAG	retention, storage	read path
RAG vs. P1-S	storage, fidelity, retrieval	**what is forgotten**
P1-S vs. P1-A	storage, selection	**fidelity**

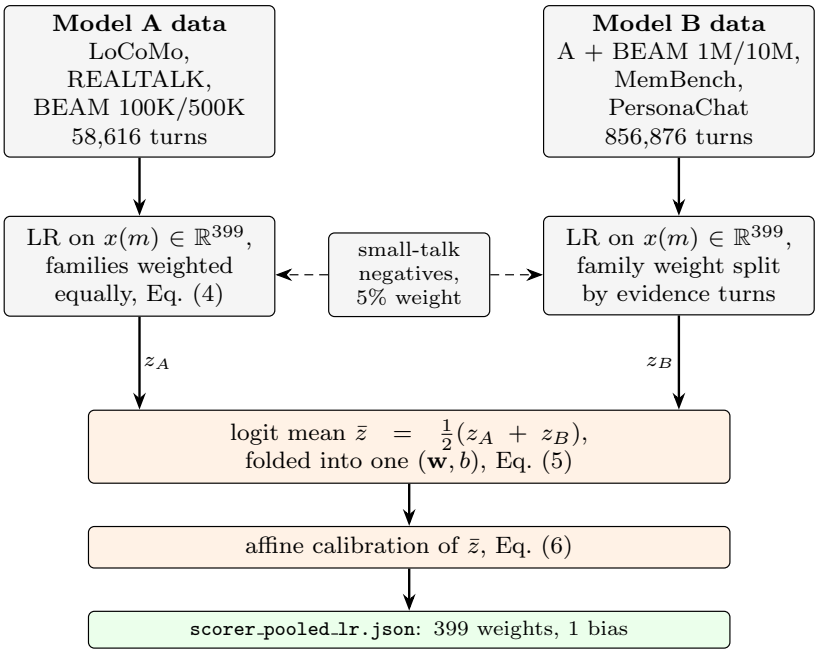


Figure 4: How the shipped scorer is built: two logistic regressions, Eq. (4), averaged in logit space and folded into one, Eq. (5), then calibrated, Eq. (6). It ships as 399 weights and a bias in plain JSON.

The whole paper leans on the second contrast. RAG and P1-S hold the same verbatim turns, in the same embedding space, retrieved the same way, under the same storage cap. They differ only in which turns they throw out when the cap binds: the oldest for RAG, the lowest-utility for P1-S. Whatever gap opens between them belongs to the decision policy.

4. Metrics

We use two retrieval-level metrics. Evidence recall, sometimes called strict recall, is the share of questions for which every gold evidence turn made it into the assembled context. Answer recall is the share for which the answer text itself survived into the context. It is a string-containment test, close in spirit to extractive QA scoring [40], and we compute it over the 715 of 1,527 questions whose answer can be read directly off the evidence turns.

The difference between the two matters a great deal. Evidence recall credits an identifier. A compressed gist keeps the identifier of a turn whose text it threw away, so it scores a hit with none of the answer behind it. During development we watched evidence recall climb from 8.1% to 14.8% while answer recall sat at 12.8%; the whole gain was bookkeeping. Dropping versus compressing is exactly the comparison where the two metrics part ways, and evidence recall alone would have pointed us to the opposite conclusion. We therefore treat answer recall as the primary metric and report evidence recall next to it, so the disagreement stays in view.

Both metrics measure context recall, which bounds end-task accuracy from above without being accuracy. What they offer is determinism: unlike LLM-judged accuracy [41], they do not mix the memory policy up with the reader's reasoning. Section 6.8 measures accuracy directly.

5. Experimental Setup

Our main corpus is LoCoMo [32]: 10 multi-session conversations, 5,882 turns and 1,527 scoreable questions. We leave out the category-5 (adversarial) items. They do carry evidence, but the right response is to decline, so retrieving that evidence would not count as success.

Retrieval and semantic supersession use all-MiniLM-L6-v2 [36] at 384 dimensions. Budgets are counted with `cl100k_base`, so every budget is an exact token count. The P2 judge is Qwen2.5-0.5B-Instruct [42], run locally. Rather than generating a rating and parsing it, we read salience as the expected value over the digit-token logits of a single forward pass. This is faster, cannot fail to parse, and leaves no ties in the eviction order.

Every interval is a paired bootstrap [43] with $B = 10{,}000$ resamples drawn by conversation, paired with an exact McNemar test [44, 45]. The resampling unit is not a detail. The 1,527 questions come from 10 conversations, so they are not 1,527 independent trials [46], and one of our own claims does not survive the correction (Section 6.3). Every model is a pinned local copy, with the model hub disabled at import, and every numeric table is generated from the committed result files rather than typed in by hand.

6. Results

6.1. Storage was never budgeted

Work in this area caps the context assembled for each query and leaves the store unbounded, and until this point so did ours. Under that accounting a store-everything baseline pays nothing to hold all 419 turns of a conversation, so forgetting can only ever lose information. When we ran our own ablation that way, it reported the tiering and compression machinery as a net loss against keeping everything and retrieving. Given how cost was being counted, that verdict was correct.

A storage budget S changes the question from what fits in the context to what is worth keeping at all. Table 1 sweeps S with the context budget fixed at 2048 tokens, and the ranking under this accounting is not the ranking under the old one. We regard this as the most consequential observation in the paper: a protocol that leaves out a cost ends up answering a different question from the one it seems to ask.

6.2. Selection pays; compression does not

Table 1 reports both metrics across the sweep. P1-S, which drops the lowest-utility item when the cap binds, leads on answer recall at every budget. P1-A, which compresses that item instead, leads on evidence recall at every budget, and yet at $S = 8192$ it falls behind even RAG on answer recall, by 3.4 points.

Here the identifier problem of Section 4 shows up in real numbers. A demoted gist keeps the evidence identifiers of a turn whose text is gone. Judged on evidence recall alone, compression looks like the better use of a bounded store; checked for whether the answer text survived, it looks like the worse one.

6.3. Statistical significance

Table 2 gives each contrast with intervals resampled by conversation. Scored eviction beats FIFO on answer recall by +9.37 points, and the interval excludes zero. The matching evidence-recall gain of +2.42 points does not survive clustering ($p = 0.08$), so we do not claim it. We still report it, because a claim that fails a correctness check says something useful about the check.

Table 1: Recall against the storage budget S in tokens, with the context budget fixed at 2048. Δ is P1-S minus RAG on answer recall: identical storage, identical fidelity, identical retrieval, differing only in which turns are forgotten.

	Evidence recall (%)			Answer recall (%)			
S	RAG	P1-A	P1-S	RAG	P1-A	P1-S	Δ
1024	4.3	9.3	5.3	9.1	11.9	**15.8**	+6.7
2048	10.1	19.4	11.8	18.3	22.7	**27.8**	+9.5
4096	17.8	31.8	20.2	28.1	30.3	**37.5**	+9.4
8192	30.6	45.6	38.6	44.3	41.0	**57.8**	+13.4
16384	53.3	58.2	57.9	66.4	63.1	**71.3**	+4.9

Table 2: Paired bootstrap ($B = 10{,}000$) resampled by conversation, with exact McNemar, at $S = 4096$. The second row is the claim we decline to make.

Comparison	Metric	Δ (pts)	95% CI (clustered)	McNemar p	
P1-S vs. RAG (FIFO)	answer	**+9.37**	[+5.45, +14.90]	2.8×10^{-5}	✓
P1-S vs. RAG (FIFO)	evidence	+2.42	[−1.48, +6.43]	0.08	n.s.
P1-S vs. P1-A (compress)	answer	**+7.13**	[+4.32, +10.28]	2.1×10^{-5}	✓
P1-S vs. P1-A (compress)	evidence	**−11.53**	[−14.73, −8.47]	1.8×10^{-23}	✓
P1-S vs. P0 (recency)	answer	**+19.16**	[+15.42, +24.13]	8.2×10^{-19}	✓
P1-S vs. P0 (recency)	evidence	**+10.15**	[+6.58, +13.84]	1.2×10^{-15}	✓

The P1-S versus P1-A rows put signs on the divergence: +7.13 on answer recall and −11.53 on evidence recall, both significant. The two metrics disagree about direction, not just about size.

6.4. The rate–distortion crossover is not reached

Rate–distortion reasoning says compression must win eventually: compress a record of everything far enough and it should beat a faithful record of only part [13]. Shrinking S cannot test this, because every policy hits the floor first, near 0.2% evidence recall at $S = 256$. The missing axis is length, so we concatenate up to ten LoCoMo conversations — 186K tokens — holding S fixed, which raises the compression ratio to 182×. Table 3 reports no crossover: the margin of selection over compression narrows but stays positive, and the best operating point stays at the vertex, a subset at full fidelity.

There is a caveat. A concatenated stream is not a natural long conversation: each question is about one of the pieces and the rest act as distractors. So this is a stress test of retention, not a long-dialogue benchmark. We rule out the crossover along this axis, up to this ratio, and claim nothing further.

6.5. The embedding index is not free

Counting storage in text tokens ignores the index that makes retrieval possible in the first place. A 384-dimensional `float32` vector occupies 1,536 B, while a LoCoMo turn averages about 130 B of text. The index therefore outweighs the content it indexes by roughly a factor of 12, and the limiting resource becomes the number of stored items rather than their length.

Table 4 repeats the sweep with the budget denominated in bytes and each retrievable item charged for its vector. Two things follow. The selection result of Section 6.2 survives the change of units: P1-S leads at every budget above 32 KiB. And below about 40 KiB

Table 3: Answer recall against compression ratio at $S = 2048$. The rightmost column is the margin of dropping over compressing.

Ratio	RAG	P1-A	P1-S	P1-S − P1-A
9×	18.3	22.7	**27.8**	+5.2
18×	10.8	13.0	**18.9**	+5.9
45×	7.8	10.3	**11.6**	+1.3
91×	4.6	7.4	**8.7**	+1.3

Table 4: Answer recall (%) with the budget denominated in bytes, charging 1,536 B per retrievable item for its embedding. P0 maintains no index and is therefore flat.

Budget (KiB)	P0	RAG	P1-A	P1-S
32	**18.3**	5.0	5.9	15.1
64	18.3	11.0	9.4	**24.9**
128	18.3	20.0	11.9	**33.1**
256	18.3	33.0	15.8	**46.0**
512	18.3	50.3	22.2	**61.0**

an embedding index cannot pay for its own storage, so P0, which keeps no index at all, does best. A protocol that counts only text never sees this regime, yet it is exactly where a memory layer on a small device would live.

6.6. Scorer analysis

If the value lies in the decision, the scorer that makes it is the part worth improving. Table 5 removes one mechanism at a time. Retrieval dominates, at −31.6 evidence points, and scored eviction is worth 6.4 points. The demotion row is marked because it shows the same divergence in miniature: removing demotion costs 11.5 evidence points and gains 7.1 answer points.

Table 6 varies only the scorer, everything else fixed at the configuration of Section 6.2. Three results follow, all significant under clustering. Scoring is worth doing: the hand-written heuristic gains +11.19 answer points over unscored recency. The LLM judge, the approach of LLM-as-a-judge evaluation [41], nonetheless loses: in the pre-fix run, Qwen2.5-0.5B-Instruct reaches 30.9% against the heuristic's 36.8%, −5.87 points with the interval excluding zero, at some fifteen minutes of GPU time per corpus against microseconds for a regular expression. And the headroom is real: a logistic scorer fitted under leave-one-conversation-out reaches 48.3%, +11.47 over the heuristic, so the judge was not limited by a ceiling that does not exist.

We think the judge fails because of what it is shown. It rates each turn on its own, but salience is not a property of a turn on its own: a phone number matters because someone asks for it later. The fitted scorer sees the same isolated text but absorbs corpus-level base rates a per-turn prompt cannot convey — its strongest single feature is turn length, ahead of every hand-designed cue. Under leave-one-conversation-out it never sees the questions of the conversation it scores, so it is deployable in the way any supervised component is: fitted offline on labelled conversations, then applied to new ones. What this does not establish is that the base rates it absorbs carry over to a different kind of conversation, which Section 6.7 tests.

Table 5: Component ablation at $S = 4096$, showing the change from the full system. †: the row on which the two metrics disagree in direction.

Component removed	Δ evidence	Δ answer
Retrieval (Tier 3)	−31.6	−28.0
Demotion on pressure †	−11.5	+7.1
Scored eviction (→ FIFO)	−6.4	−7.4
Informative compressor	−4.4	−1.7
Consolidation merge	−3.4	−4.8
Semantic supersession	−0.3	−0.3

Table 6: Single-component scorer swap at $S = 4096$ with context 2048. Policy, retrieval and compression are fixed and only the scorer varies. P3 is fitted on the other nine conversations. Pre-fix run (Section 8).

	What the scorer sees	Evid.	Ans.	Δ vs. P1	95% CI
P0	position only	12.6	25.6	−11.19	[−17.24, −6.19]
P1	regex + shallow NER	19.8	**36.8**	—	—
P2	the turn, in isolation	14.0	30.9	−5.87	[−10.77, −1.76]
P3	turn + fitted base rates	33.4	48.3	+11.47	[+4.90, +17.86]

6.7. A learned scorer across two corpora

LoCoMo's dialogues are generated by an LLM, so a scorer fitted to them may learn the generator's style rather than what people later ask about. We test this on REALTALK [34]: ten conversations between real people over 21 days, 8,944 turns and 660 questions with turn-level evidence in LoCoMo's schema, of which 160 are answer-recoverable. The scorer is a logistic regression over 15 shallow features of the message (length, digits, dates, capitalisation, question and filler cues) concatenated with its 384-dimensional MiniLM embedding, labelled by whether any question cites the turn as evidence. Every held-out conversation is scored by three models that never saw it: trained on the other conversations of the same corpus, on the other corpus only, or on all 19 other conversations pooled. Policy, budgets and retrieval are those of Section 6.2 with the fixed heuristic (Section 8); only the scorer varies.

Table 7 reports the result. On LoCoMo the pooled scorer reaches 52.0% answer recall against the heuristic's 37.5%, a gain of +14.55 points, and pooling beats same-corpus training by +3.36 ([+0.27, +6.52]), so adding a different corpus helped rather than diluted. Transfer works in one direction only. Trained on REALTALK alone, the scorer still beats the heuristic on LoCoMo by +9.09; trained on LoCoMo alone, it falls below the heuristic on REALTALK (−4.38, n.s.) and is significantly worse than same-corpus training there (−10.62, [−18.31, −3.45]). At this budget no gain over the heuristic on REALTALK is significant, which with 160 questions in ten clusters is a statement about power as much as about the scorer.

$S = 4096$ is the budget of the headline comparison, not one chosen for this scorer, so Table 8 repeats the pooled run at four other budgets. On LoCoMo the gain is significant from $S = 2048$ to 16384 and largest at 4096; it shrinks at both ends, where the store holds too little for any ranking to matter or nearly everything anyway. On REALTALK the gain is significant at $S = 8192$ (+16.88, [+6.31, +26.72]) and 16384. The one budget at which neither corpus shows a significant gain is $S = 1024$, about 5% of a conversation.

The embedding is what carries across. With the hand features alone, the pooled

Table 7: Learned scorer on held-out conversations of two corpora, hand features plus MiniLM, $S = 4096$, context 2048. Policy P1-S; only the scorer varies. Δ is answer recall against P1, with 95% intervals resampled by conversation; bold excludes zero.

Scorer (trained on)	AUC	Evid.	Ans.	Δ vs. P1	95% CI
LoCoMo (715 answer-recoverable questions)					
RAG (FIFO, no scorer)	–	17.8	28.1	—	—
P1 heuristic	0.599	20.2	37.5	—	—
P3, same corpus	0.793	32.9	48.7	**+11.19**	[+4.72, +17.06]
P3, other corpus only	0.692	30.1	46.6	**+9.09**	[+4.53, +13.46]
P3, pooled (shipped)	0.791	34.0	52.0	**+14.55**	[+10.40, +19.70]
REALTALK (160 answer-recoverable questions)					
RAG (FIFO, no scorer)	–	10.5	16.9	—	—
P1 heuristic	0.624	12.6	32.5	—	—
P3, same corpus	0.737	18.5	38.8	+6.25	[−3.01, +13.42]
P3, other corpus only	0.681	11.2	28.1	−4.38	[−12.50, 0.00]
P3, pooled (shipped)	0.749	17.3	36.2	+3.75	[−2.92, +8.65]

Table 8: The pooled learned scorer (P3) against the P1 heuristic across the store budget S, answer recall (%), context 2048, same protocol as Table 7. Δ is P3 minus P1 with 95% intervals resampled by conversation; bold excludes zero.

	LoCoMo			REALTALK		
S	P1	P3	Δ [95% CI]	P1	P3	Δ [95% CI]
1024	15.8	19.0	+3.22 [0.00, +6.40]	8.1	10.0	+1.88 [−1.95, +5.36]
2048	27.8	33.7	**+5.87** [+1.83, +9.88]	16.2	15.0	−1.25 [−10.62, +3.98]
4096	37.5	52.0	**+14.55** [+10.40, +19.70]	32.5	36.2	+3.75 [−2.92, +8.65]
8192	57.8	69.1	**+11.33** [+6.06, +16.41]	41.2	58.1	**+16.88** [+6.31, +26.72]
16384	71.3	77.2	**+5.87** [+2.04, +10.56]	61.3	68.1	**+6.88** [+2.12, +12.21]

scorer is significantly *worse* than the heuristic on REALTALK (-8.75, $[-17.45, -2.27]$): its feature weights encode LoCoMo's style, such as long turns ending in a period, which people messaging each other do not write. This is the same failure the heuristic shows on LongMemEval (Section 9). Logistic regression was kept after comparing it with boosted trees and a 64-unit MLP on the same folds: neither beat it by more than the spread between held-out conversations, and the trees were much slower both to fit and on the write path.

6.8. End-task accuracy

Everything so far is context recall, which caps end-task accuracy without measuring it. To measure it, we pass each assembled context to one fixed reader, Qwen2.5-1.5B-Instruct-4bit decoding greedily for at most 32 tokens, and score the output with the same normaliser the answer-recall metric applies to the context. "The answer reached the context" and "the answer reached the output" are then one predicate evaluated at two points in the pipeline, so the ceiling comparison is exact rather than approximate. The run covers all 715 answer-recoverable questions with two controls: a closed-book arm that receives no memory at all, and an oracle arm that receives exactly the cited evidence.

Table 9 reports the outcome. The closed-book floor is 1.7%, so parametric knowledge contributes almost nothing on this benchmark and the margin above that floor is

Table 9: End-task accuracy through one fixed reader at $S = 4096$ with context 2048, over all 715 answer-recoverable questions. Acc.|present is accuracy restricted to the questions whose answer reached the context; it is undefined for the arms that never place one. Pre-fix run (Section 8).

Arm	Acc.	F1	Ans. present	Acc.|present
Closed-book *(floor)*	1.7	5.0	0.0	–
P0 sliding window	8.3	11.9	18.3	40.5
RAG store-all	12.4	17.7	28.1	42.3
P1-A adaptive (compress)	13.1	19.9	30.1	42.3
P1-S select (drop)	**18.5**	24.7	36.8	49.4
Oracle *(ceiling)*	67.3	75.7	100.0	67.3

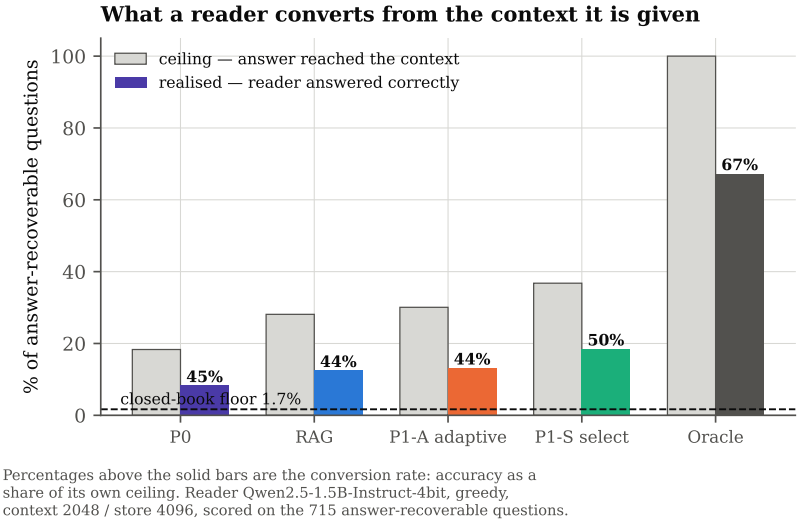


Figure 5: Reader accuracy by memory policy, with the closed-book floor and the oracle ceiling. The ordering matches the recall ordering, at a lower level.

attributable to the memory layer. P1-S reaches 18.5% against RAG's 12.4%. Resampling by conversation as before, the realised gap is +6.0 points with a 95% interval of [+3.3, +9.1], which is 69% of the +8.7-point recall gap between the same two arms in this pre-fix run (Section 8).

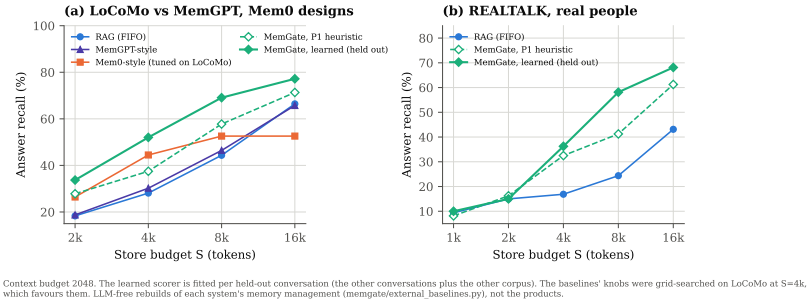
Accuracy is not bounded by answer recall, since a reader may guess, so the table separates the two regimes. Accuracy given that the answer was present is 49.4% for P1-S against 42.3% for RAG, while accuracy given that it was absent is below 1% for every arm. The selection advantage therefore arrives through both available routes: P1-S places the answer in context more often, and the contexts it assembles are also the ones the reader uses more successfully. The oracle arm converts a complete context into a correct answer 67.3% of the time, which bounds what any memory policy can deliver through this reader and locates the remaining headroom in the reader rather than in the memory layer.

6.9. Against MemGPT-style and Mem0-style memory

So far every baseline has been our own. To compare MemGate with designs that people actually use, we rebuilt the memory management of MemGPT [8] and Mem0 [16] inside the same harness, at the same context budget and store budgets. The MemGPT-style arm keeps a recent queue and a searchable archive, and folds messages evicted from the archive into a recursive summary that is always in context and loses its oldest detail

Table 10: Answer recall (%) on LoCoMo against rebuilt MemGPT-style and Mem0-style memory, context 2048. The baselines' settings were tuned on LoCoMo at $S = 4096$; P1-S and the held-out learned scorer P3 were not. Bold Δ: 95% interval resampled by conversation excludes zero.

		MemGPT	Mem0			Δ vs. Mem0-style	
S	RAG	-style	-style	P1-S	P3	P1-S	P3
2048	18.3	18.7	26.4	27.8	**33.7**	+1.4	**+7.3**
4096	28.1	30.2	44.5	37.5	**52.0**	**−7.0**	**+7.6**
8192	44.3	46.4	52.6	57.8	**69.1**	+5.2	**+16.5**
16384	66.4	65.7	52.6	71.3	**77.2**	**+18.7**	**+24.6**



Context budget 2048. The learned scorer is fitted per held-out conversation (the other conversations plus the other corpus). The baselines' knobs were grid-searched on LoCoMo at S=4k, which favours them. LLM-free rebuilds of each system's memory management (memgate/external_baselines.py), not the products.

Figure 6: Answer recall against the store budget for MemGate and the rebuilt MemGPT-style and Mem0-style memories (Table 10).

first. The Mem0-style arm writes one compressed fact per turn with ADD, UPDATE (the nearest stored fact above a similarity threshold is replaced, newest wins) and NOOP for filler, and evicts the least recently touched fact. Neither calls an LLM: summaries and facts are written by MemGate's own extractive compressor, so these are the systems' memory structures rather than their prompts. In the baselines' favour, each one's settings were grid-searched on LoCoMo itself at $S = 4096$ (MemGPT-style: summary share 0.25, 16-token gists; Mem0-style: 32-token facts, update threshold 0.65). No MemGate arm was tuned on these answers.

Table 10 and Fig. 6 report the result. The learned scorer beats both rebuilds at every budget, by +7.3 to +24.6 points over Mem0-style and +11.5 to +22.7 over MemGPT-style, every gap significant under clustering. The untrained P1 heuristic beats MemGPT-style everywhere (+5.6 to +11.3) but loses to the tuned Mem0-style at $S = 4096$, the budget its settings were chosen on, by 7.0 points with the interval excluding zero; it is level at 2048, ahead at 8192 by +5.2 (McNemar $p = 0.03$, but not significant once resampled by conversation), and well ahead at 16384. Mem0-style stops improving at 52.6% from $S = 8192$: UPDATE replaces a fact with its nearest neighbour, so a threshold loose enough to help at small S discards distinct facts that merely look alike, and the store never fills. MemGPT-style stays within about two points of plain RAG, because its summary holds gists whose answers were compressed away, the failure of Section 6.2 in a different form.

6.10. Chats of up to ten million tokens

LoCoMo conversations average about 19K tokens. BEAM [47] supplies synthetic user–assistant chats at 100K, 500K, 1M and 10M tokens with probing questions that cite turns, and MemBench [48] short agent trajectories made of factual statements. Two repairs were needed before BEAM 1M and 10M could be scored: some chats restart their turn numbering partway, so each question is pinned to the segment whose cited turns share the

Table 11: Held-out answer recall (%) at each corpus's own store budget, 5-fold by conversation. $P3_{12}$ is the PR #12 recipe (LoCoMo, REALTALK, BEAM 100K/500K), refitted without the held-out fold; $P3_{ship}$ is the shipped ensemble of Section 6.11, under the same protocol. n is the number of answer-recoverable questions. Bold: the 95% interval, resampled by conversation, excludes zero. ‡PersonaChat validation dialogues, never trained on; queries are the persona lines, a proxy described in Section 6.11.

Corpus	S	n	RAG	P1	$P3_{12}$	$P3_{12}$−RAG [95% CI]	$P3_{ship}$	$P3_{ship}$−$P3_{12}$ [95% CI]
LoCoMo	4096	715	28.1	37.5	50.3	**+22.2** [+17.5, +27.6]	51.9	+1.5 [−0.2, +3.4]
REALTALK	4096	160	16.9	32.5	36.2	**+19.4** [+7.5, +26.4]	36.2	0.0 [−3.3, +3.1]
BEAM 100K	26K	34	32.4	35.3	73.5	**+41.2** [+14.3, +67.6]	73.5	0.0 [0.0, 0.0]
BEAM 500K	26K	55	32.7	32.7	63.6	**+30.9** [+16.7, +44.8]	61.8	−1.8 [−10.0, +5.8]
BEAM 1M	26K	52	13.5	11.5	50.0	**+36.5** [+21.8, +51.0]	50.0	0.0 [−7.3, +8.0]
BEAM 10M	26K	13	7.7	23.1	53.8	**+46.2** [+10.0, +76.9]	38.5	−15.4 [−44.4, 0.0]
MemBench	2048	367	76.8	97.5	95.6	**+18.8** [+14.2, +23.7]	97.5	**+1.9** [+0.3, +3.8]
Everyday chat‡	1024	1762	28.8	30.4	61.4	—	69.0	**+7.6** [+5.9, +9.5]

most words with it, and only when that segment leads clearly. At 1M, 3 of 625 questions could not be pinned; at 10M, 128 of 200 are pinned and the rest are abstentions or cite turns that no part of the chat holds. Each conversation is then scored by a learned scorer fitted, under five folds by conversation, without it; BEAM 1M and 10M are outside its training corpora altogether. The store budget is $S = 26{,}000$ for BEAM, about 2.5% of a 1M-token chat.

Table 11 reports answer recall. The learned scorer beats RAG on all seven corpora with the interval excluding zero, by +36.5 points on BEAM 1M and +46.2 on BEAM 10M. On a million-token chat RAG keeps only the most recent turns, so almost nothing it is asked about survives: evidence recall, which covers 622 questions on BEAM 1M rather than 52, is 22.2% for the learned scorer against 0.9% for RAG. The one corpus on which it is not the best is MemBench, whose turns are short factual statements, which is what the P1 rules were written to find; there P1 leads it by 1.9 answer points (n.s.). BEAM answers are mostly paraphrases, so only a few questions per corpus are answer-recoverable, and the BEAM 10M row rests on 13 questions.

6.11. Ten times the training data

The learned scorer that the application first shipped was fitted on 58,616 turns. We retrained it on ten times that: BEAM 1M (74,630 turns) and 10M (208,696), and MemBench's observation and participation sets, 594,000 turns in all, with the families of Eq. (4) weighted equally. Each candidate was replayed through P1-S on held-out conversations at three to five budgets per corpus, and the whole protocol was repeated with the folds dealt from three different orders, 26,444 replays in all. The rule for shipping a model is that it must lose nowhere.

More data on its own did not help (Table 12, first column). The 10x model gains on MemBench, the corpus that was added, but loses evidence recall on BEAM 1M by 5.1 to 5.6 points and on BEAM 500K by 3.0 to 5.2, with the interval excluding zero in every shuffle. Our reading is that a linear model on 399 inputs was not short of data but of capacity: the new turns are a different kind of chat, mostly negatives (BEAM 10M adds 208,696 turns and only a few hundred evidence labels), and they pull the weights toward short factual statements at the cost of BEAM's long answers. Boosted trees, the obvious source of capacity, gained on BEAM 500K but lost 8.5 points on LoCoMo ($[-11.1, -5.3]$) and were rejected.

Three changes put it right. The family weight of Eq. (4) is split among corpora by

Table 12: Retraining: change against the PR #12 recipe in three reshuffled fold orders, at each corpus's budget (Table 11), held out. "App path" adds the application's chunking, guard and multi-query retrieval. Bold: the 95% interval, resampled by conversation, excludes zero.

	10x (594K turns)	Shipped (857K turns)	
Corpus	Evidence	Answer	Evidence
LoCoMo	−0.4, −0.8, −0.2	+1.5, −0.3, +0.3	+0.1, −0.2, +0.2
REALTALK	+0.1, −1.6, 0.0	0.0, 0.0, +0.6	−0.3, −0.8, +0.2
BEAM 100K	−1.3, −0.8, **−3.7**	0.0, +2.9, −2.9	+1.8, **+1.8**, +0.6
BEAM 500K	**−5.2**, **−4.4**, **−3.0**	−1.8, +1.8, +1.8	+1.6, +2.0, **+2.8**
BEAM 1M	**−5.1**, **−5.6**, **−5.3**	0.0, −1.9, +1.9	+0.3, 0.0, −0.4
BEAM 10M	**−2.3**, −2.5, −0.5	−15.4, 0.0, −7.7	−1.2, −1.8, +1.2
MemBench	**+5.5**, **+5.8**, **+6.2**	**+1.9**, **+1.9**, **+2.2**	**+5.6**, **+5.9**, **+6.4**
Everyday chat‡	—	**+7.6**, **+8.2**, **+8.0**	**+8.8**, **+9.1**, **+9.3**
App path, BEAM 100K	**+6.1**, +4.2, +6.0	**+11.8**, **+11.8**, +8.8	**+12.4**, **+11.5**, **+11.9**
App path, LoCoMo	−0.5, −0.7, −0.2	+1.5, 0.0, +0.4	+0.3, +0.1, +0.5
App path, REALTALK	+0.7, +0.3, −0.3	+3.1, +1.9, +0.6	+0.5, −0.1, 0.0

evidence turns instead of equally, so BEAM 10M's many unlabelled turns count for little. Everyday chat is added as a fifth family: 894 long chats assembled from PersonaChat [49], 262,876 turns, weakly labelled (a turn is evidence when it states one of its speaker's persona lines). And the result is averaged in logit space with the earlier model, Eq. (5), which keeps the ranking that suited BEAM's long answers while adding what the new chats taught, at a linear model's cost. The shipped model is trained on 856,876 turns, 14.6 times the original. For an everyday-chat test, 50 PersonaChat validation chats that no model saw are replayed at $S = 1024$; each stated persona line is asked as a query, and the answer is the words of it that the person said. That is a proxy, since query and evidence share wording, but it treats every arm alike.

At the paper's budgets the shipped model has no significant loss on any corpus in any of the three shuffles (Table 12). It gains on everyday chat (+7.6 to +8.2 answer points, 69.0% against 61.4%), on MemBench, and on BEAM 100K through the application's own path (+11.5 to +12.4 evidence points), and ties on BEAM 1M, where the 10x model lost. Across all other budgets it has one significant loss: BEAM 1M evidence recall at $S = 8000$, -1.7 $[-2.8, -0.6]$, about one cited turn in sixty.

7. From Harness to Application

The policy is not only an experiment; it also runs as a working system. A chat application wraps a reader model in the same `observe()` and `build_context()` calls the experiments use (Fig. 1). Every message, from the user or from the model, passes through the policy, and every question is answered from the context MemGate assembles, never from the raw history.

The application adds one step the experiments do not have: an entry gate that scores each message the moment it is sent. A message the scorer labels filler (a greeting, "ok cool", at most two words), or a model reply scoring below 0.1, is forgotten straight away and never takes up space. Under P1-A and P1-S a message with $u \geq \tau_{\text{fact}}$ is also written to Tier 3 immediately, with its score, while staying in Tier 1 for the next few replies; when it later leaves Tier 1, the near-duplicate rule folds it into that copy. All other messages

Table 13: Whole BEAM chats typed into the application one message at a time, $S = 26{,}000$. N chats of the given mean length in millions of tokens; n answerable questions; answer recall (%). BEAM 1M is outside the scorer's training corpora; BEAM 500K is inside them.

Chats	N	M tok.	n	P0	RAG	MemGate	Δ vs. RAG [95% CI]	McNemar p
BEAM 1M	33	1.07	52	9.6	19.2	**51.9**	**+32.7** [+19.2, +46.3]	9.1×10^{-4}
BEAM 500K	33	0.52	55	12.7	30.9	**65.5**	**+34.5** [+19.6, +49.1]	3.1×10^{-4}

follow the experimental path unchanged. The gate can be turned off, and the built-in benchmark described below runs the policies without it.

The reader can be swapped freely because it only ever receives two strings: the assembled context and the current message. By default it runs offline on an 8 GB laptop with Qwen2.5-1.5B-Instruct [42]. A hosted model (Claude, or Gemini [50] on its free tier) can take its place, and if the hosted model is rate-limited and a local copy is installed, Qwen answers that message instead. With no model at all, the application shows the context the reader would have received. P1-S is the default policy, using the shipped learned scorer of Section 6.11 inside GuardedScorer (Eqs. (5)–(7), Fig. 2). If the MiniLM embedder is missing, the application falls back to the P1 heuristic instead of scoring with the wrong features. Any text can be added. Pasted text, PDF, Word, PowerPoint and other files are cleaned, and anything longer than 120 tokens is cut into pieces of at most 48 tokens, each scored, stored and retrieved like a turn, because a single embedding of a long document describes little more than its opening.

Reusing the experimental code gives two useful properties. First, persistence is exact. A conversation is saved as its messages and restored by replaying them through `observe()`, which is deterministic, so the tiers after a restart match the tiers before it. Second, the application can check itself. A built-in benchmark recomputes the RAG and P1-S arms on all of LoCoMo at $S = 4096$ on a CPU and compares them with the committed results, reproducing 28.1% and 37.5% exactly. A side panel shows each message's utility u, the tier it went to or the reason it was forgotten, and the exact context sent to the reader, so every decision the paper measures in aggregate can be inspected one message at a time. The application runs at the experimental budgets by default. It also offers a reduced scale (a Tier 1 of four messages, a 512-token context and $S = 150$) at which eviction becomes visible within a few messages. These sizes and the entry gate are the only settings that differ from the experiments.

7.1. Stress and scale through the application

The benchmarks above replay conversations straight through the policy. To test the system a person would actually use, whole BEAM chats were typed into the application one message at a time, exactly as its page sends them, with each chat's recorded assistant turns as the replies, at $S = 26{,}000$. Each answerable question was then asked as a message, and the notes each memory would hand the reader were checked. Table 13 reports the result: on million-token chats, which MemGate holds at about 2.5%, the answer reaches the reader for 51.9% of questions against 19.2% for RAG and 9.6% for a sliding window. The store never exceeded S, and a message took 0.49 s on average on a shared 4-core machine. This run used the PR #12 scorer, the one shipped at the time; repeating it with the current one takes about ten hours and was not done, but the held-out application-path rows of Table 12 are the current model's.

Much of the gap comes down to where the evidence sits. Table 14 splits LoCoMo and

Table 14: Answer recall (%) by the position of a question's evidence in the conversation, $S = 4096$. RAG and P1 are untrained. *The shipped PR #12 scorer, in-sample on both corpora.

Corpus	Evidence in	n	RAG	P1	P3*
LoCoMo	first	262	9.2	33.2	51.9
	middle	225	14.2	38.2	53.8
	last	228	63.6	41.7	54.4
REALTALK	first	86	8.1	19.8	33.7
	middle	45	11.1	40.0	55.6
	last	29	51.7	58.6	65.5

REALTALK questions by the position of their evidence in the conversation. RAG forgets oldest-first, so for evidence in the first third it keeps the answer 8–9% of the time; the untrained P1 heuristic keeps 20–33%, and the learned scorer more, although on these two corpora it is in-sample and its column is a ceiling. For evidence in the last third RAG is competitive, since recency is then the right rule.

People rarely ask one clean question per message. When each LoCoMo question was buried in 150–450 words of other chat, using the whole message as the query dropped answer recall from 53.1% to 30.3%; joining the message's questions into one query recovered 42.1%, and making each question its own query with interleaved rankings (Eq. (11)) recovers 49.8% (REALTALK: 47.5%, 26.2%, 42.5% and 43.8%).

The same tests found six input-handling bugs, each now pinned by a test: half of an emoji in one message made every later save of that chat fail; a 300 KB Word file that unzips to 300 MB used 3.8 GB of memory; deeply nested JSON dropped the connection; request bodies had no size limit; a message of only invisible characters was accepted; and a locked PDF gave an unclear error. A load test found one more: under bursts, the server's accept queue of five reset 314 of 2,000 requests from 200 simultaneous visitors. With a queue of 256 the same test completes with no errors in 25 s (95th percentile 2.7 s per message), and 100 messages sent into one chat at the same moment are all kept. A single chat of 5,000 messages stays at 4,093 of its 4,096 tokens, at 0.12 s per message by the end. The one slow path left is reopening a saved million-token chat, which replays it through memory and takes 70 s.

8. Threats to Validity

Three errors changed our results before we caught them, and in each case an invariant caught it, not a read-through of the code. Two of the three are forms of leakage, a well-known source of irreproducible machine-learning results [51, 52].

A policy that read its evaluation label. LoCoMo marks every evidence turn with a `fact_id`, and an early consolidation rule used it to decide what to promote. In effect the policy was reading the answer key. It was worth 45 to 47 recall points and invalidated the first result we reported. The fix is an invariant rather than a convention: strip every evaluation label from the input, and the resulting store must be bit-identical. Every decision point added since then was put under that check before it produced a number.

A component's value depends on the current bottleneck. We first measured the sentence embedder at +0.2 points and decided it hardly mattered. That was true only where we measured it, with retrieval saturated against a write path that threw away 93% of the conversation. Once that bottleneck was gone, the same swap was worth +19.7. An

ablation run against a bottlenecked pipeline will keep undervaluing whatever would matter once the dominant term moves.

A failure invisible in the aggregate. Batched judge scoring returned `NaN` for short turns, and only for short turns. We suspected numerical precision, but `float32` did not help. The real cause was left-padding, which produced attention rows that were entirely masked. Without an explicit guard, every filler turn would have scored `NaN` and turned the eviction order upside down.

We found a fourth defect after the main runs. The heuristic scored each turn with its speaker tag still attached, which stopped the filler rule from firing and handed almost every turn the proper-noun bonus: 5,863 of 5,882 LoCoMo turns were labelled facts. The recall tables and Table 7 use the fixed scorer; the judge, end-task, abstractive and LongMemEval runs came before the fix. Fixing it moves the headline P1-S result from 36.8% to 37.5%, and shrinks the learned scorer's lead over the heuristic at $S = 2048$ from +5.2 to +2.7 points, since part of that lead was the defect holding the heuristic back.

9. Limitations

Few conversations. The conversation is the right unit for resampling, but it leaves us with few clusters and therefore wide intervals. Effects that do not survive are reported as not significant. REALTALK adds ten conversations but only 160 answer-recoverable questions, too few to settle most of its differences.

A single reader model. Section 6.8 uses one 1.5B four-bit model [42] with greedy decoding. We claim the ordering of the policies, not their absolute accuracies. The oracle arm's 67.3% shows that much of the remaining error belongs to the reader. A larger reader would lift every arm and could narrow or widen the margins, and it may also use position in the context differently [5].

Compression was tested in both forms. Replacing P1-A's word selection with a 1.5B model that rewrites each evicted turn is worth +4.06 answer points (CI $[+0.84, +6.66]$), so part of the negative result was down to the compressor rather than to compression as such. Selection still wins (pre-fix run): 36.8% against 34.1%, at 2.1 s against 3,892 s and 4,879 generations. The claim therefore narrows: a better compressor helps, but it does not overtake dropping.

Concatenated streams are synthetic, as discussed in Section 6.4.

The scorer is tuned to this corpus. On a 94-item LongMemEval [6] subsample, whose turns average 210 tokens against LoCoMo's 32, the heuristic's separation between evidence and non-evidence turns flipped sign, and scored eviction lost to FIFO. The +9.37 of Section 6.2 should be read as what a policy recovers *given a scorer suited to the corpus*, not as a property of scored selection in general. We have since tested the learned scorer held out on BEAM, where the heuristic also ranks evidence below chance, and on MemBench and everyday chat (Section 6.10). LongMemEval, where the heuristic's sign first flipped, remains untested with it because we could not fetch it in our environment. The scorer's labels are distant supervision (a turn is positive if any question cites it), and that supervision is twice as dense on LoCoMo (24% of turns) as on REALTALK (12%). On PersonaChat it is weaker still: a word-overlap match with a persona line.

Few answerable questions at scale. BEAM's answers are mostly paraphrases, so only 34 to 55 questions per scale are answer-recoverable, and 13 at 10M. The intervals in Table 11 are wide as a result, which is why we also report evidence recall, covering about ten times as many questions. The rebuilt MemGPT-style and Mem0-style arms reproduce

those systems' memory structures without their LLM calls, so they measure the designs rather than the products.

Single embedder and single judge. The negative result of Section 6.6 is evidence about a 0.5B model prompted one turn at a time, not about LLM-judged salience in general. A larger judge, or one shown the neighbouring turns, has not been tried, and it is the most direct way to challenge the finding.

10. Conclusion

Under the accounting this field normally uses, tiered compression loses to storing everything and retrieving. That result is an artefact of leaving the store unbounded. Once the store is bounded, deciding which items to forget is worth +9.37 answer-recall points at identical storage, fidelity and retrieval, while compressing an evicted item instead of dropping it costs 7.13. The point at which rate–distortion theory [12] says compression should start to pay is not reached by 182×. Counting the embedding index in the budget reverses the ranking below about 40 KiB.

The obvious route to a better scorer, an LLM judge, loses to a regular expression, and for a reason we can name: salience is not a property of a turn read on its own. A logistic scorer over a sentence embedding, trained on generated and human conversations together, beats the heuristic by +14.55 points on held-out LoCoMo conversations and by a significant margin at every store budget from 2048 to 16384 tokens. It transfers from human dialogue to generated dialogue, but not the other way round. Through a fixed reader, +6.0 of the pre-fix +8.7 points survive as correct answers. The same scorer beats MemGPT-style and Mem0-style memory, rebuilt in this harness and tuned in their favour, at every budget we tried. On held-out chats of up to ten million tokens it beats retrieval on every corpus, and when million-token chats are typed into the application it keeps the answer 51.9% of the time against 19.2%. Ten times the training data did not, on its own, produce a better linear scorer. Weighting corpora by their labels, adding everyday chat and averaging with the earlier model did, with no significant loss at the paper's budgets.

Our practical advice is narrow: keep a scored subset at full fidelity rather than compressing everything [18, 21], and count the budget in bytes with the index included. The methodological lesson is broader. Three of the results above changed sign or size once a measurement error was removed, and none of those errors showed up in the aggregate numbers they distorted.

Two questions stay open. The pooled scorer's gains on human conversation become significant only once the store holds about 40% of a conversation, and the scorer is untested on LongMemEval, where the hand-written cues inverted. More human-written evidence, rather than a larger model, is what would settle both. Second, a scorer learned online from retrieval feedback is already statistically level with the heuristic on LoCoMo, with no labels and no future questions (−2.94, n.s.). Whether it can start from the pooled scorer and then adapt to each user is the step from a scorer fitted to benchmarks to one fitted to the person using the application.

Acknowledgment

The authors thank the faculty of the School of Technology Management & Engineering, SVKM's NMIMS, Indore Campus, for their guidance throughout this capstone project.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability. The code, the result files behind every table and figure, the trained scorer and the chat application are available at `https://github.com/Nixx5555/Mem`. LoCoMo, REALTALK, BEAM, MemBench and PersonaChat are public datasets available from their authors.

Author contributions (CRediT). Yash Ramchandani: Investigation, Validation, Writing – review & editing. Simar Singh Khanuja: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft. Praveen Modi: Supervision, Writing – review & editing.

Use of generative AI. During the preparation of this work the authors used an AI assistant for drafting and code assistance. The authors reviewed and edited the content and take full responsibility for it.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: Proc. Int. Conf. Learning Representations (ICLR), 2023.

[2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, J.-R. Wen, A survey on large language model based autonomous agents, Frontiers of Computer Science 18 (6) (2024) 186345.

[3] T. B. Brown, B. Mann, N. Ryder, et al., Language models are few-shot learners, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 1877–1901.

[4] Chroma Research, Context rot: How increasing input tokens impacts LLM performance, Tech. rep., Chroma (2025).

[5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Transactions of the Association for Computational Linguistics 12 (2024) 157–173.

[6] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, D. Yu, LongMemEval: Benchmarking chat assistants on long-term interactive memory, arXiv preprint arXiv:2410.10813 (2024).

[7] T. R. Sumers, S. Yao, K. Narasimhan, T. L. Griffiths, Cognitive architectures for language agents, Transactions on Machine Learning Research (2024).

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez, MemGPT: Towards LLMs as operating systems, arXiv preprint arXiv:2310.08560 (2023).

[9] W. Zhong, L. Guo, Q. Gao, H. Ye, Y. Wang, MemoryBank: Enhancing large language models with long-term memory, in: Proc. AAAI Conf. Artificial Intelligence, Vol. 38, 2024, pp. 19724–19731.

[10] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, J.-R. Wen, A survey on the memory mechanism of large language model based agents, arXiv preprint arXiv:2404.13501 (2024).

[11] C. E. Shannon, Coding theorems for a discrete source with a fidelity criterion, in: IRE National Convention Record, Vol. 4, 1959, pp. 142–163.

[12] T. M. Cover, J. A. Thomas, Elements of Information Theory, 2nd Edition, Wiley, 2006.

[13] R. Colaco, A. Lahjouji, A rate–distortion framework for memory in language agents, arXiv preprint arXiv:2607.08032 (2026).

[14] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: Proc. ACM Symp. User Interface Software and Technology (UIST), 2023.

[15] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, Y. Zhang, A-MEM: Agentic memory for LLM agents, in: Advances in Neural Information Processing Systems (NeurIPS), 2025.

[16] P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav, Mem0: Building production-ready AI agents with scalable long-term memory, arXiv preprint arXiv:2504.19413 (2025).

[17] R. C. Atkinson, R. M. Shiffrin, Human memory: A proposed system and its control processes, Psychology of Learning and Motivation 2 (1968) 89–195.

[18] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu, LLMLingua: Compressing prompts for accelerated inference of large language models, in: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 13358–13376.

[19] J. Mu, X. L. Li, N. Goodman, Learning to compress prompts with gist tokens, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[20] A. Chevalier, A. Wettig, A. Ajith, D. Chen, Adapting language models to compress contexts, in: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023.

[21] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, F. Wei, In-context autoencoder for context compression in a large language model, in: Proc. Int. Conf. Learning Representations (ICLR), 2024.

[22] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis, Efficient streaming language models with attention sinks, in: Proc. Int. Conf. Learning Representations (ICLR), 2024.

[23] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, B. Chen, H2O: Heavy-hitter oracle for efficient generative inference of large language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[24] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, T. P. Lillicrap, Compressive transformers for long-range sequence modelling, in: Proc. Int. Conf. Learning Representations (ICLR), 2020.

[25] Y. Wu, M. N. Rabe, D. Hutchins, C. Szegedy, Memorizing transformers, in: Proc. Int. Conf. Learning Representations (ICLR), 2022.

[26] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, F. Wei, Augmenting language models with long-term memory, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023.

[27] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 9459–9474.

[28] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[29] S. Borgeaud, A. Mensch, J. Hoffmann, et al., Improving language models by retrieving from trillions of tokens, in: Proc. Int. Conf. Machine Learning (ICML), Vol. 162 of PMLR, 2022, pp. 2206–2240.

[30] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101.

[31] P. J. Denning, The working set model for program behavior, Communications of the ACM 11 (5) (1968) 323–333.

[32] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang, Evaluating very long-term conversational memory of LLM agents, in: Proc. Annu. Meeting Assoc. Computational Linguistics (ACL), 2024.

[33] Y. Bai, X. Lv, J. Zhang, et al., LongBench: A bilingual, multitask benchmark for long context understanding, in: Proc. Annu. Meeting Association for Computational Linguistics (ACL), 2024, pp. 3119–3137.

[34] D.-H. Lee, A. Maharana, J. Pujara, X. Ren, F. Barbieri, REALTALK: A 21-day real-world dataset for long-term conversation, arXiv preprint arXiv:2502.13270 (2025).

[35] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, M. Zhou, MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 5776–5788.

[36] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2019.

[37] M. Mintz, S. Bills, R. Snow, D. Jurafsky, Distant supervision for relation extraction without labeled data, in: Proc. Joint Conf. 47th Annu. Meeting ACL and 4th IJCNLP, 2009, pp. 1003–1011.

[38] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825–2830.

[39] J. C. Platt, Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods, in: Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.

[40] P. Rajpurkar, J. Zhang, K. Lopyrev, P. Liang, SQuAD: 100,000+ questions for machine comprehension of text, in: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2016, pp. 2383–2392.

[41] L. Zheng, W.-L. Chiang, Y. Sheng, et al., Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, Vol. 36, 2023.

[42] Qwen Team, Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2025).

[43] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[44] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (2) (1947) 153–157.

[45] T. G. Dietterich, Approximate statistical tests for comparing supervised classification learning algorithms, Neural Computation 10 (7) (1998) 1895–1923.

[46] A. C. Cameron, J. B. Gelbach, D. L. Miller, Bootstrap-based improvements for inference with clustered errors, Review of Economics and Statistics 90 (3) (2008) 414–427.

[47] M. Tavakoli, et al., Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs, in: International Conference on Learning Representations (ICLR), 2026, arXiv:2510.27246.

[48] H. Tan, et al., MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents, in: Findings of the Association for Computational Linguistics: ACL 2025, 2025, arXiv:2506.21605.

[49] S. Zhang, E. Dinan, J. Urbanek, A. Szlam, D. Kiela, J. Weston, Personalizing dialogue agents: I have a dog, do you have pets too?, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), 2018, pp. 2204–2213.

[50] Gemini Team, Google, Gemini: A family of highly capable multimodal models, arXiv preprint arXiv:2312.11805 (2023).

[51] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data 6 (4) (2012) 1–21.

[52] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804.